

Lista de Recursividade — Respostas

Soluções em Python

1. MDC (Algoritmo de Euclides)

Se b é zero, o MDC é a. Caso contrário, $\text{MDC}(a, b) = \text{MDC}(b, a \bmod b)$.

```
def mdc(a, b):
    a, b = abs(a), abs(b)
    return a if b == 0 else mdc(b, a % b)

print(mdc(48, 18)) # 6
```

2. Potência x^n

Caso base: $n = 0$ retorna 1. Para n negativo, calcula-se o inverso. Usa exponenciação rápida (divide o expoente por 2 a cada chamada).

```
def potencia(x, n):
    if n == 0:
        return 1
    if n < 0:
        return 1 / potencia(x, -n)
    metade = potencia(x, n // 2)
    return metade * metade if n % 2 == 0 else metade * metade * x

print(potencia(2, 10)) # 1024
```

3. Inverter string

Caso base: string com 0 ou 1 caractere. Passo: inverte o restante e coloca o primeiro caractere no final.

```
def inverter(s):
    return s if len(s) <= 1 else inverter(s[1:]) + s[0]

print(inverter("python")) # nohtyp
```

4. Palíndromo

Remove espaços e converte para minúsculas. Compara o primeiro e o último caractere e repete para o miolo da string.

```
def palindromo(s):
    s = s.replace(" ", "").lower()
    if len(s) <= 1:
        return True
    return s[0] == s[-1] and palindromo(s[1:-1])

print(palindromo("A arara")) # True
```

5. Soma dos dígitos

Caso base: número com um dígito. Passo: último dígito ($n \% 10$) mais a soma dos dígitos do restante ($n // 10$).

```
def soma_digitos(n):
    return n if n < 10 else n % 10 + soma_digitos(n // 10)

print(soma_digitos(1234)) # 10
```

6. Anagramas

Fixa cada caractere na primeira posição e gera recursivamente os anagramas do restante. O conjunto (set) elimina duplicatas quando há letras repetidas.

```
def anagramas(s):
    if len(s) <= 1:
        return {s}
    resultado = set()
    for i, c in enumerate(s):
        for resto in anagramas(s[:i] + s[i+1:]):
            resultado.add(c + resto)
    return resultado

print(sorted(anagramas("abc")))
# ['abc', 'acb', 'bac', 'bca', 'cab', 'cba']
```